```
Finished in 0.02619 seconds (files took 0.21264 seconds to load)
8 examples, 0 failures

Randomized with seed 21495
```

Try booting the app with rackup again, and use curl to make a few requests:

```
$ curl localhost:9292/expenses --data '{"payee":"Zoo", "amount":10,
"date":"2017-06-10"}' -w "\n"
{"expense_id":1}
$ curl localhost:9292/expenses --data '{"payee":"Starbucks", "amount":7.5,
"date":"2017-06-10"}' -w "\n"
{"expense_id":2}
$ curl localhost:9292/expenses/2017-06-10 -w "\n"
[{"id":1,"payee":"Zoo","amount":10.0,"date":"2017-06-10"},{"id":2,"payee":"St
arbucks","amount":7.5,"date":"2017-06-10"}]
```

It works! Buy yourself a glass of your favorite beverage, commit your work, and then let's wrap up in the next section. Just don't forget to record the expense.

Your Turn

In this chapter, you implemented the final piece of the app: the storage layer that writes to a real database. You wrote *integration specs* to test this layer. Because these specs made changes to the same global state (the database), they could interfere with one another. You used RSpec's random test ordering and --bisect ability to unearth these dependencies. You fixed them with a clean around hook, and kept noisy database transaction code out of your integration specs.

Once your integration specs were passing, you found that your end-to-end specs were also green. You've completed the first major piece of a real app.

During this project, you've come to know RSpec quite well. You've learned how to test individual methods using expectations and test doubles. You've created example groups and shared test data to keep your specs laser-focused on what the code is supposed to be doing. You've used RSpec's spec runner to unearth and fix problems in your test code.

Over the next few parts of the book, we're going to take a deep dive into each of these aspects of RSpec. But first, try your hand at an exercise or two.

Exercises

In these exercises, you're going to flesh out your specs so that they check your code's behavior more closely. Then, you'll use outside-in development to add a new feature (specifically, support for a new data format) to your expense tracker.